09-KServe

1 KServe

Document Version: 1.0
Generated: December 4, 2025
Standard: KServe InferenceService CRD
Status: Industry Standard (Linux Foundation - Kubeflow)

1.1 Overview

KServe is a Kubernetes-native framework for serverless inference. It provides abstractions for deploying, managing, and autoscaling machine learning models on Kubernetes.

1.1.1 Key Features

- **InferenceService CRD**: Kubernetes Custom Resource Definition
- **Serverless**: Auto-scaling with KNative
- **Multi-Framework**: TensorFlow, PyTorch, ONNX, etc.
- **Predictors/Transformers/Explainers**: Composable pipeline
- **Traffic Splitting**: Canary deployments
- **V2 Protocol**: Standard inference protocol

1.1.2 Primary Use Cases

1. **Serverless Inference**: Auto-scaling model serving
2. **Model Versioning**: Canary deployments
3. **Explanations**: Model interpretability
4. **Feature Transformation**: Pre-processing pipeline
5. **Multi-Model Serving**: Multiple models per service

1.2 Authoritative References

- **KServe Docs**: https://kserve.github.io/website
- **GitHub Repository**: https://github.com/kserve/kserve
- **InferenceService CRD**: https://kserve.github.io/website/reference/api
- **V2 Protocol**: https://kserve.github.io/website/reference/api/v2_protocol

---

1.3 Format Structure

1.3.1 InferenceService YAML Specification

Real example (inference-service.yaml):

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: iris-classifier
  namespace: kserve-demo
  annotations:
    serving.kserve.io/deploymentMode: "Serverless"
spec:
  # Model name for inference
  predictor:
    serviceAccountName: kserve-service-account

    # Sklearn predictor
    sklearn:
```

```yaml
      storageUri: "gs://my-bucket/iris-model"
      resources:
        limits:
          cpu: "1"
          memory: "2Gi"
        requests:
          cpu: "500m"
          memory: "1Gi"

    # Autoscaling
    minReplicas: 1
    maxReplicas: 10

    # Timeout and concurrency
    timeoutSeconds: 60
    containerConcurrency: 10

    # Probes
    livenessProbe:
      httpGet:
        path: /v2/models/iris-classifier
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 30

    readinessProbe:
      httpGet:
        path: /v2/models/iris-classifier/ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10

  # Pre-processing transformer
  transformer:
    serviceAccountName: kserve-service-account
    custom:
      spec:
        containers:
          - name: transformer
            image: my-registry/iris-transformer:latest
            env:
              - name: PREDICTOR_HOST
                value: iris-classifier-predictor-default
              - name: PROTOCOL
                value: v2
            resources:
              limits:
                cpu: "500m"
                memory: "512Mi"
              requests:
                cpu: "250m"
                memory: "256Mi"

  # Model explainer
  explainer:
    serviceAccountName: kserve-service-account
    alibi:
      storageUri: "gs://my-bucket/iris-explainer"
      resources:
        limits:
          cpu: "1"
          memory: "2Gi"
        requests:
          cpu: "500m"
          memory: "1Gi"

  # Traffic routing
  canaryTrafficPercent: 10

  # External traffic
  predictor:
    canaryRevisionName: iris-classifier-v2
    canaryTrafficPercent: 10
```

```yaml
status:
  # URL for inference
  url: "http://iris-classifier.kserve-demo.svc.cluster.local"

  # Conditions
  conditions:
    - type: PredictorReady
      status: "True"
      reason: "MinimumReplicasAvailable"
    - type: TransformerReady
      status: "True"
    - type: ExplainerReady
      status: "True"
    - type: IngressReady
      status: "True"
    - type: Ready
      status: "True"
```

1.3.2 V2 Protocol Request/Response

Request Format (JSON):

```json
{
  "id": "request-123",
  "parameters": {
    "sequence_id": 1,
    "sequence_start": false,
    "sequence_end": false,
    "priority": 5,
    "timeout_micros": 0
  },
  "inputs": [
    {
      "name": "input_tensor",
      "shape": [1, 4],
      "datatype": "FP32",
      "parameters": {},
      "data": [5.1, 3.5, 1.4, 0.2]
    }
  ]
}
```

Response Format (JSON):

```
{
  "model_name": "iris-classifier",
  "model_version": "v1",
  "id": "request-123",
  "parameters": {},
  "outputs": [
    {
      "name": "prediction",
      "shape": [1, 3],
      "datatype": "FP32",
      "parameters": {},
      "data": [0.98, 0.01, 0.01]
    },
    {
      "name": "class_id",
      "shape": [1, 1],
      "datatype": "INT64",
      "data": [0]
    }
  ]
}
```

1.4 Procedural Use-Cases

1.4.1 Use-Case 1: Deploy Scikit-Learn Model

Python Training & Upload:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn import datasets
import joblib
import os

# Train model
iris = datasets.load_iris()
model = RandomForestClassifier(n_estimators=10)
model.fit(iris.data, iris.target)

# Save
os.makedirs("./iris-model", exist_ok=True)
joblib.dump(model, "./iris-model/model.joblib")

# Upload to cloud storage (GCS)
import subprocess
subprocess.run([
    "gsutil", "-m", "cp", "-r",
    "./iris-model",
    "gs://my-bucket/"
])
```

KServe Deployment:

```
kubectl apply -f - <<EOF
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: iris-classifier
  namespace: kserve-demo
spec:
  predictor:
    sklearn:
      storageUri: "gs://my-bucket/iris-model"
EOF
```

Send Prediction Request:

```
import requests
import json

# Get service URL
service_url = "http://iris-classifier.kserve-demo.svc.cluster.local"

# Prepare request (V2 protocol)
request_body = {
    "inputs": [
        {
            "name": "input",
            "shape": [1, 4],
            "datatype": "FP32",
            "data": [5.1, 3.5, 1.4, 0.2]
        }
    ]
}

# Send prediction
response = requests.post(
    f"{service_url}:8080/v2/models/iris-classifier/infer",
    json=request_body
)

predictions = response.json()
print(f"Prediction: {predictions['outputs'][0]['data']}")
```

1.4.2 Use-Case 2: PyTorch Model with Transformer

Create Transformer Service:

```
# transformer.py
from typing import Dict
import json
import base64
import numpy as np
from PIL import Image
from io import BytesIO

class ImageTransformer:
    """Pre-processes images for model input"""

    def preprocess(self, request: Dict) -> Dict:
        """Convert image to model input"""

        # Extract image from request
        image_data = request["instances"][0]["image_b64"]
        image_bytes = base64.b64decode(image_data)
        image = Image.open(BytesIO(image_bytes))
```

```
        # Resize to model input size (224x224)
        image = image.resize((224, 224))
        image_array = np.array(image, dtype=np.float32)

        # Normalize (ImageNet mean/std)
        mean = np.array([0.485, 0.456, 0.406])
        std = np.array([0.229, 0.224, 0.225])
        image_array = (image_array / 255.0 - mean) / std

        # Add batch dimension
        image_array = np.expand_dims(image_array, axis=0)

        # Return V2 format
        return {
            "inputs": [
                {
                    "name": "input_image",
                    "shape": list(image_array.shape),
                    "datatype": "FP32",
                    "data": image_array.flatten().tolist()
                }
            ]
        }

    def postprocess(self, response: Dict) -> Dict:
        """Convert model output to user format"""

        predictions = response["outputs"][0]["data"]
        top_k = 5

        # Get top-k predictions
        top_indices = np.argsort(predictions)[-top_k:][::-1]

        return {
            "predictions": [
                {
                    "class_id": int(idx),
                    "class_name": self.get_class_name(idx),
                    "confidence": float(predictions[idx])
                }
                for idx in top_indices
            ]
        }

    def get_class_name(self, class_id: int) -> str:
        # ImageNet class names
        classes = {
            0: "tench", 1: "goldfish", 2: "great_white_shark"
            # ... 1000 classes
        }
        return classes.get(class_id, "unknown")
```

KServe with Transformer:

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: image-classifier
spec:
  predictor:
    pytorch:
      storageUri: "gs://my-bucket/resnet50-model"
      runtimeVersion: "latest"

  transformer:
    custom:
      spec:
        containers:
          - name: transformer
            image: my-registry/image-transformer:latest
            env:
              - name: PREDICTOR_HOST
                value: image-classifier-predictor-default
```

1.4.3 Use-Case 3: Canary Deployment

Blue-Green Update:

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: model-service
spec:
  predictor:
    serviceAccountName: kserve-account

    # Current version (90% traffic)
    pytorch:
      storageUri: "gs://models/model-v1"
      minReplicas: 2
      maxReplicas: 10

    # New version (10% traffic for testing)
    canaryTrafficPercent: 10

    canaryRevisionName: model-service-v2
```

Complete Canary Spec:

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: bert-classifier
spec:
  predictor:
    # Production version
    pytorch:
      storageUri: "gs://models/bert-v1.1"
      minReplicas: 3
      maxReplicas: 50
      env:
        - name: BATCH_SIZE
          value: "32"

    canaryTrafficPercent: 15

    # Canary version metadata
    canaryRevisionName: bert-classifier-v2-candidate

---
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: bert-classifier-v2-candidate
spec:
  predictor:
    pytorch:
      storageUri: "gs://models/bert-v2-rc1"
      minReplicas: 1
      maxReplicas: 10
      env:
        - name: BATCH_SIZE
          value: "64"
```

1.5 Examples

1.5.1 Example 1: TensorFlow Serving Integration

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: tf-model
spec:
  predictor:
    tensorflow:
      storageUri: "gs://models/saved-model"
      runtimeVersion: "2.9.0"
      resources:
        requests:
          memory: "2Gi"
          cpu: "1"
        limits:
          memory: "4Gi"
          cpu: "2"

    minReplicas: 2
    maxReplicas: 10

    # KNative autoscaling
    scaleMetric: "rps"
    scaleTarget: 1000
```

1.5.2 Example 2: ONNX Runtime Predictor

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: onnx-bert
spec:
  predictor:
    onnx:
      storageUri: "gs://models/bert.onnx"
      resources:
        limits:
          memory: "1Gi"
          cpu: "500m"
```

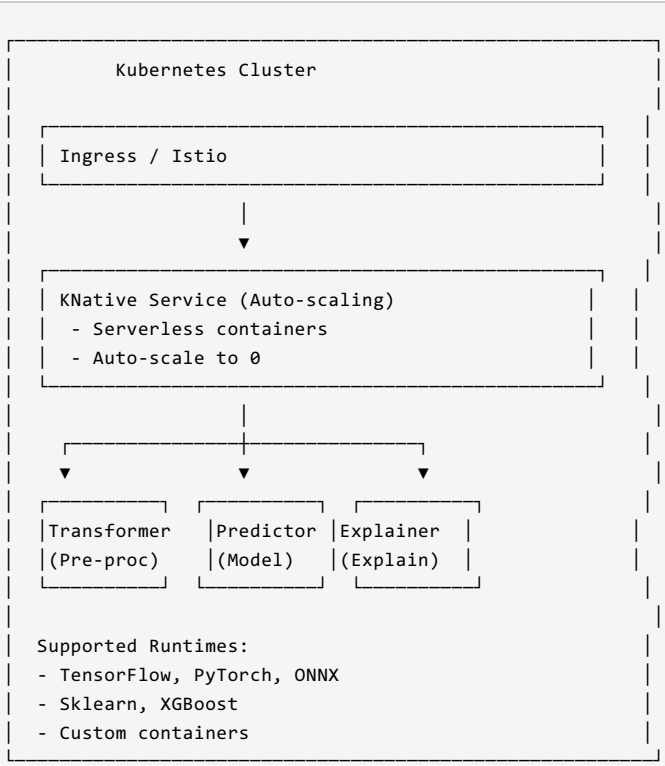
1.5.3 Example 3: Custom Predictor with Python

```python
# predictor.py
from kserve import Model, ModelServer
from typing import Dict
import torch

class CustomBERTPredictor(Model):
    def __init__(self, name: str):
        super().__init__(name)
        self.name = name
        self.ready = False

    def load(self):
        self.model = torch.jit.load("/mnt/models/bert.pt")
        self.model.eval()
        self.ready = True

    def predict(self, request: Dict) -> Dict:
        instances = request["instances"]

        # Batch process
        outputs = []
        for instance in instances:
            input_ids = torch.tensor(instance["input_ids"]).unsqueeze(0)

            with torch.no_grad():
                logits = self.model(input_ids)

            predictions = torch.softmax(logits, dim=1)
            outputs.append(predictions.tolist())

        return {"predictions": outputs}

if __name__ == "__main__":
    model = CustomBERTPredictor("bert-classifier")
    model.load()
    ModelServer().start([model])
```

Deploy Custom Predictor:

```yaml
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: custom-bert
spec:
  predictor:
    custom:
      spec:
        containers:
          - name: predictor
            image: my-registry/bert-predictor:latest
            ports:
              - containerPort: 8080
            resources:
              requests:
                memory: "2Gi"
                cpu: "1"
              limits:
                memory: "4Gi"
                cpu: "2"
```

1.6 KServe Architecture Diagram



1.7 Tools & Ecosystem

Tool	Description	URL
KServe	Main framework	https://kserve.github.io
Kubeflow	ML platform	https://www.kubeflow.org
KNative	Serverless	https://knative.dev
Istio	Service mesh	https://istio.io
Prometheus	Monitoring	https://prometheus.io

Navigation: Back to Index | Previous: gRPC Serving | Next: Triton Inference Server